

CSL351 : COMPUTER NETWORKS

Assignment - 6

Name - Ashish Ranjan

ID no. - 12340370

GDrive Link :

<https://drive.google.com/drive/folders/1xLqnY8lQXYa39WEDqKEZggffsp9zn6JF?usp=sharing>

Instruction Regarding the GDrive Link :

For each part different folder with different sub folder regarding codes, gnuplots ,shellscripts, pcap are present.

File Structure of the Zip :

Two separate folders :

PartA contains :

as6_que1.cc as the main ns3 code, .dat for the output data generated used by the gnuplot code as graph.gnuplot.

three png(flowgood.png , flowloss.png , flowthroughput.png) for the output generated by the graph.gnuplot.

the routing-tables.txt as mentioned in the assignment.

PartA.xml which is used by the netanim for showing visually.

PartB contains:

as6_que2.cc as the main ns3 code for PartB, graphs_que2.gnuplot for creating the gnuplot, .dat files(outputdataBit.dat , outputdataByte.dat , outputdataPacket.dat) used by the gnuplot code for creating the graph.

three png(erpacket.png , erbit.png , erpacket.png) for the output generated by the graphs_que2.gnuplot.

runner.sh shell script used for running multiple simulations.

Report From the next page :

Part A :

COMMAND USED :

`./ns3 build`

`./ns3 run as6_que1`

Output Terminal :

```
flow id: 1
source: 10.1.1.1 -> destination: 10.1.4.1
throughput: 2874.13 Kbps
goodput: 4825.94 Kbps
lost packets: 7
=====
flow id: 2
source: 10.1.3.1 -> destination: 10.1.7.1
throughput: 1069.27 Kbps
goodput: 1867.23 Kbps
lost packets: 0
=====
flow id: 3
source: 10.1.8.1 -> destination: 10.1.8.1
throughput: 0.291667 Kbps
goodput: 5.125 Kbps
lost packets: 0
=====
flow id: 5
source: 10.1.2.1 -> destination: 10.1.6.1
throughput: 1979.87 Kbps
goodput: 1874.47 Kbps
lost packets: 0
=====
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/ns-allinone-3.44/ns-3.44$
```

Explanation :

For this Part we established a multi hop network environment within NS3 as (/PartA/as6_que1.cc) Our primary objective was to evaluate data transmission across a core routing backbone by a mix of TCP and UDP traffic and subsequently analyzing the network's performance using built-in tools.

Network Infrastructure & Topology

- **Device Instantiation:** We constructed the physical architecture using a NodeContainer, which deploys eight user terminals (termi) and three core interconnected routers (routers).
- **Properties:** To simulate realistic network constraints, PointToPointHelper was utilized to define link characteristics. Edge terminals connect to the routers via access links (10Mbps bandwidth,

2ms latency). In contrast, the router to router connections are intentionally bottlenecked, (5Mbps and 3Mbps) and higher delays (5ms and 10ms) to force potential network congestion.

Protocol Configuration & Routing

- Every simulated device is equipped with standard networking capabilities via the `InternetStackHelper`, allowing them to process ip based traffic.
- IP spaces are mapped out using `Ipv4AddressHelper`. The edge devices receive dynamic subnet assignments via an iterative loop (formatting subnets like 10.1.X.0), while the central links are assigned fixed, dedicated IPv4 blocks (10.1.10.0 and 10.1.20.0).
- Seamless communication across these subnets is achieved by invoking `Ipv4GlobalRoutingHelper::PopulateRoutingTables()`, which automatically generates the routing logic necessary for packets to navigate the topology.

Workload Simulation & Traffic Generation

- The simulation relies on specific sender receiver pairings. We deployed `ns3::TcpSocketFactory` for reliable data streams on ports 8770, 8790, and 8800, and `ns3::UdpSocketFactory` for connectionless traffic on port 8780. Target IP addresses were dynamically retrieved from the interface containers.
- client applications act as the traffic generatrs. They are configured to push constant bit rate traffic using standard 1024 byte packet sizes, activating transmissions between the 2nd and 10th seconds of the simulation clock.

Data Analysis

- For analysis, we exported multiple diagnostic files: an XML layout for NetAnim, raw packet captures (.pcap) for Wireshark inspection, and a localized text dump (routing-tables.txt) of the router pathfinding tables.

- **Metrics:** FlowMonitor serves as our primary evaluation engine. By parsing the FlowStats, we calculated three critical metrics, outputting the results to both the console and a .dat file:

1. **Throughput:** The gross delivery rate of raw data, calculated by converting total received bytes (rxBytes) against the active duration of the flow.

Formula used:

$$\text{Throughput (Kbps)} = (\text{rxBytes} * 8.0) / \text{duration} / 1024$$

2. **Goodput:** The net delivery rate of usable payload. The script intelligently adjusts for protocol overhead, subtracting 40 bytes for TCP headers (984 bytes) and 28 bytes for UDP headers (996 bytes) from the initial 1024 byte frame to find the true payload size.

Formula used:

$$\text{Goodput (Kbps)} = (\text{rxPackets} * \text{payloadSize} * 8.0) / \text{duration} / 1024$$

3. **Packet Drops:** A direct count of undelivered traffic (lostPackets).

Formula used:

$$\text{Packets Lost} = \text{flowStats.lostPackets}$$

Inferences

- The spatial visualization and generated routing tables confirm that the node creation and IPv4 subnetting successfully bridged the terminal devices across the multi hop core backbone.
- The distinct metric outputs for ports 8770 through 8800 prove that both the TCP and UDP generators successfully injected traffic, with the recorded packet loss directly highlighting the bottlenecks created by the slower central router links.

Gnuplot Script (as graph.gnuplot):

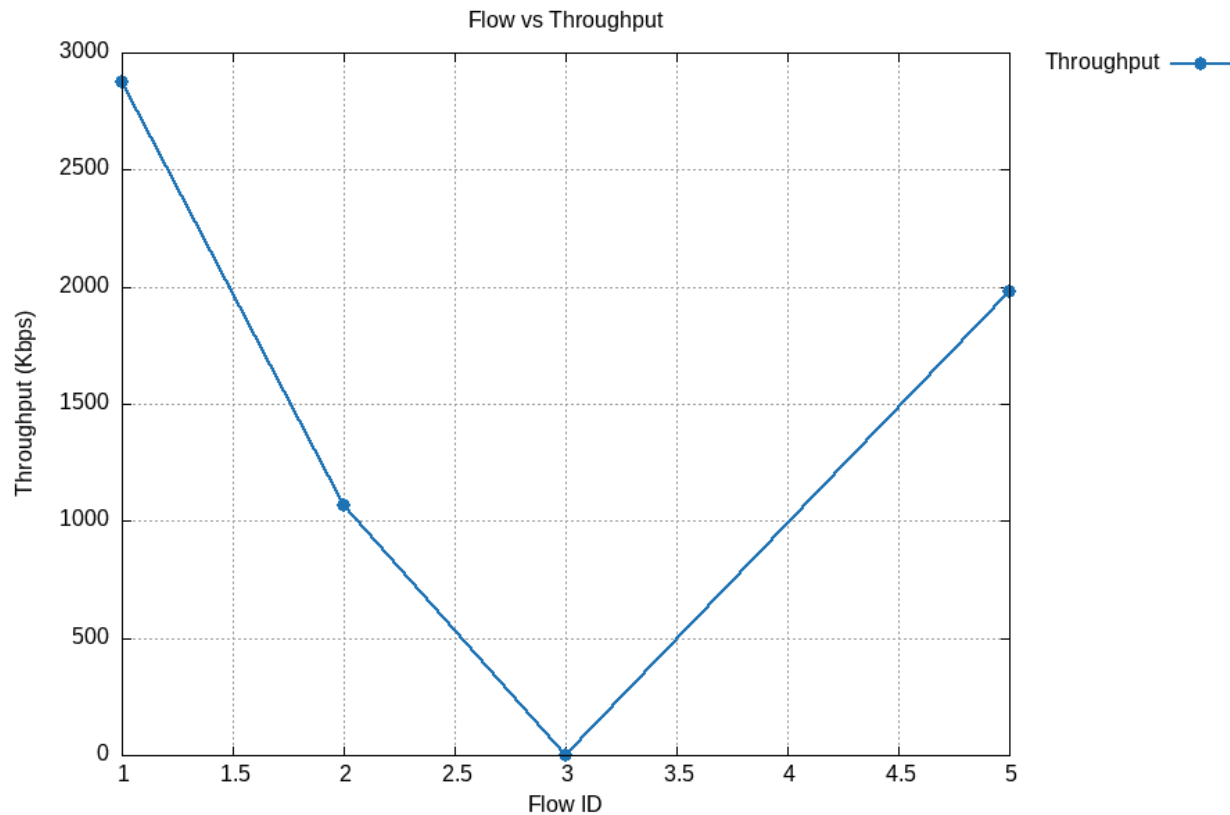
```
1 set terminal png size 900,600
2 set datafile separator whitespace
3 set grid
4 set key outside
5 set xlabel "Flow ID"
6
7
8 set output "flowthroughput.png"
9 set title "Flow vs Throughput"
10 set ylabel "Throughput (Kbps)"
11 plot "flowresults.dat" using 1:2 with linespoints lw 2.5 pt 7 ps 1.2 lc rgb "#1f77b4" title "Throughput"
12
13 set output "flowgoodput.png"
14 set title "Flow vs Goodput"
15 set ylabel "Goodput (Kbps)"
16 plot "flowresults.dat" using 1:3 with linespoints lw 2.5 pt 7 ps 1.2 lc rgb "#2ca02c" title "Goodput"
17
18 set output "flowloss.png"
19 set title "Flow vs Total Packets Lost"
20 set ylabel "Packets Lost"
21 plot "flowresults.dat" using 1:4 with linespoints lw 2.5 pt 7 ps 1.2 lc rgb "#d62728" title "Packets Lost"
22
23
24
25
26
27
28
```

To visualize the individual network flow performance, I wrote a Gnuplot named as **graph.gnuplot** script configured to parse data from the **flowresults.dat** file. I established a standard 900x600 resolution, added gridlines. The script systematically generates three distinct charts Throughput, Goodput, and Total Packets Lost by mapping the Flow ID against their respective data columns.

Question 1 :

COMMAND USED : `gnuplot graph.gnuplot`

We receive the graph of Flow v/s Throughput (as **flowthroughput.png**) :



Observations:

- Flow 1 recorded the maximum throughput, approximately 2900 Kbps, with Flow 5 in second about 2000 Kbps.
- Flow 2 maintained a medium throughput level registering 1100 Kbps.
- Flow 3 failed to transmit any data successfully, resulting in a throughput of 0 Kbps.

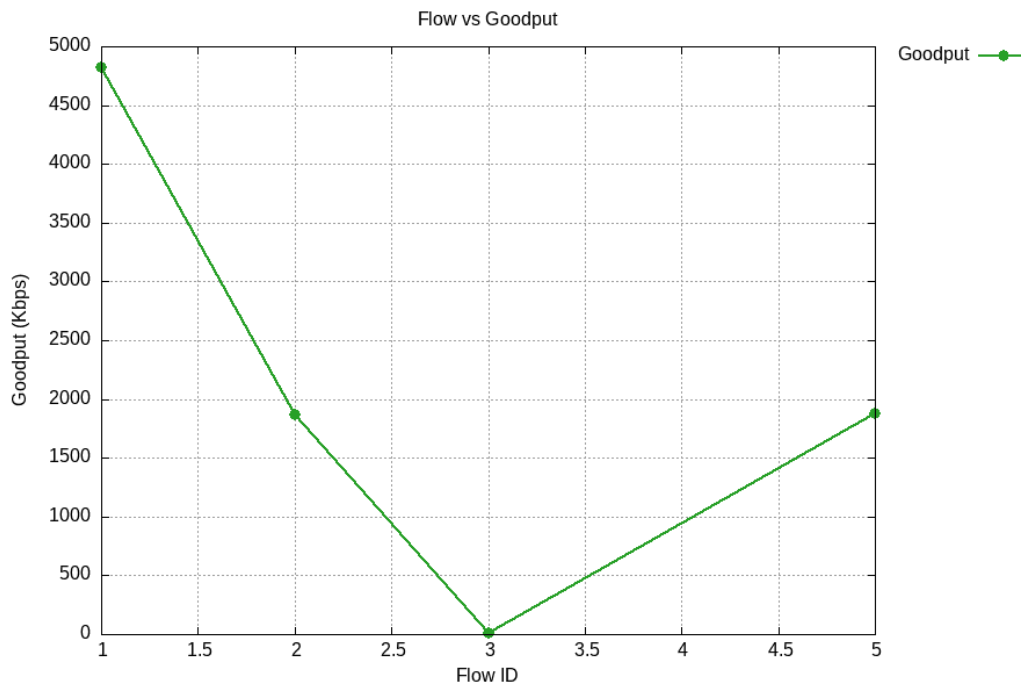
Inferences :

- The observed variations in throughput for Flows 1, 2, and 5 corresponds to the distinct application level data rates at 1 Mbps, 2 Mbps, 3 Mbps configured within the network topology.
- Bottleneck links between the routers actively restricted the flows, preventing them from achieving the maximum rates established at the edge nodes.
- The total absence of data transmission for Flow 3 (0 Kbps) suggests a significant issue, such as a critical network bottleneck, severe congestion, or a complete packet drop along its specific route.

Question 2:

COMMAND USED : `gnuplot graph.gnuplot`

We receive the graph of Flow v/s Goodput (as **flowgoodput.png**):



Observations :

- Flow 1 achieved the peak goodput, maximizing at approximately 4800 Kbps.
- Flows 2 and 5 have comparable and moderate goodput values, around 1900 Kbps.
- Flow 3 resulted in a goodput of 0 Kbps.

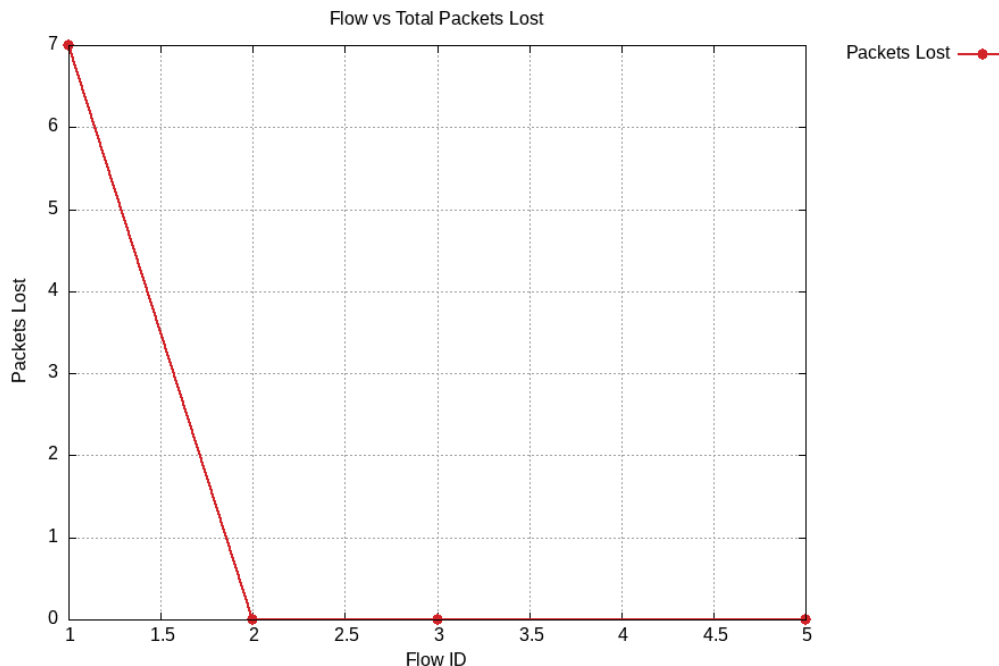
Inferences :

- The identical goodput measured for Flows 2 and 5 suggests they are subject to similar network bottleneck conditions or have been configured with matching application data rates at the source.
- The 0 Kbps goodput for Flow 3 validates the total failure previously observed in the throughput graph, which mean a 100% packet loss rate.

Question 3:

COMMAND USED : `gnuplot graph.gnuplot`

We receive the graph of Flow v/s Total packet lost (**as flowloss.png**):



Observations :

- Flow 1 recorded packet loss, with 7 packets dropped during the simulation.
- Flows 2, 3, and 5 completed transmission with zero dropped packets.

Inferences :

- The packet loss observed in Flow 1 indicates router buffer overflow, which is likely caused by its high transmission rate consuming the bottleneck link's capacity.
- Flows 2 and 5 operated well within the network's bandwidth, resulting in reliable, lossless data transmission.
- Flow 3 shows zero packet loss because it experienced a total failure in data transmission.

Question 4 :

The **routing-tables.txt** file was generated by initializing an ns3 output stream directed to the specified file name. For each of the three router nodes, we have accessed its underlying IPv4 routing protocol interface and then called the built in PrintRoutingTable() method, using the output stream as an argument to dump the entire routing entries directly into the text file.

Output Terminal :

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/ns-allinone-3.44/ns-3.44$ gnuplot graph1gnuplot
Node: 8, Time: +0s, Local time: +0s, Ipv4ListRouting table
  Priority: 0 Protocol: ns3::Ipv4StaticRouting
Node: 8, Time: +0s, Local time: +0s, Ipv4StaticRouting table


| Destination | Gateway | Genmask       | Flags | Metric | Ref | Use | Iface |
|-------------|---------|---------------|-------|--------|-----|-----|-------|
| 127.0.0.0   | 0.0.0.0 | 255.0.0.0     | U     | 0      | -   | -   | 0     |
| 10.1.1.0    | 0.0.0.0 | 255.255.255.0 | U     | 0      | -   | -   | 1     |
| 10.1.2.0    | 0.0.0.0 | 255.255.255.0 | U     | 0      | -   | -   | 2     |
| 10.1.3.0    | 0.0.0.0 | 255.255.255.0 | U     | 0      | -   | -   | 3     |
| 10.1.10.0   | 0.0.0.0 | 255.255.255.0 | U     | 0      | -   | -   | 4     |
| 10.1.20.0   | 0.0.0.0 | 255.255.255.0 | U     | 0      | -   | -   | 5     |


  Priority: -10 Protocol: ns3::Ipv4GlobalRouting
Node: 8, Time: +0s, Local time: +0s, Ipv4GlobalRouting table


| Destination | Gateway   | Genmask         | Flags | Metric | Ref | Use | Iface |
|-------------|-----------|-----------------|-------|--------|-----|-----|-------|
| 10.1.1.1    | 10.1.1.1  | 255.255.255.255 | UH    | -      | -   | -   | 1     |
| 10.1.2.1    | 10.1.2.1  | 255.255.255.255 | UH    | -      | -   | -   | 2     |
| 10.1.3.1    | 10.1.3.1  | 255.255.255.255 | UH    | -      | -   | -   | 3     |
| 10.1.10.2   | 10.1.10.2 | 255.255.255.255 | UH    | -      | -   | -   | 4     |
| 10.1.4.2    | 10.1.10.2 | 255.255.255.255 | UH    | -      | -   | -   | 4     |
| 10.1.5.2    | 10.1.10.2 | 255.255.255.255 | UH    | -      | -   | -   | 4     |
| 10.1.6.2    | 10.1.10.2 | 255.255.255.255 | UH    | -      | -   | -   | 4     |
| 10.1.20.2   | 10.1.20.2 | 255.255.255.255 | UH    | -      | -   | -   | 5     |
| 10.1.7.2    | 10.1.20.2 | 255.255.255.255 | UH    | -      | -   | -   | 5     |
| 10.1.8.2    | 10.1.20.2 | 255.255.255.255 | UH    | -      | -   | -   | 5     |
| 10.1.4.1    | 10.1.10.2 | 255.255.255.255 | UH    | -      | -   | -   | 4     |
| 10.1.5.1    | 10.1.10.2 | 255.255.255.255 | UH    | -      | -   | -   | 4     |
| 10.1.6.1    | 10.1.10.2 | 255.255.255.255 | UH    | -      | -   | -   | 4     |
| 10.1.7.1    | 10.1.20.2 | 255.255.255.255 | UH    | -      | -   | -   | 5     |
| 10.1.8.1    | 10.1.20.2 | 255.255.255.255 | UH    | -      | -   | -   | 5     |
| 10.1.1.0    | 10.1.1.1  | 255.255.255.0   | UG    | -      | -   | -   | 1     |
| 10.1.2.0    | 10.1.2.1  | 255.255.255.0   | UG    | -      | -   | -   | 2     |
| 10.1.3.0    | 10.1.3.1  | 255.255.255.0   | UG    | -      | -   | -   | 3     |
| 10.1.10.0   | 10.1.10.2 | 255.255.255.0   | UG    | -      | -   | -   | 4     |
| 10.1.4.0    | 10.1.10.2 | 255.255.255.0   | UG    | -      | -   | -   | 4     |
| 10.1.5.0    | 10.1.10.2 | 255.255.255.0   | UG    | -      | -   | -   | 4     |
| 10.1.6.0    | 10.1.10.2 | 255.255.255.0   | UG    | -      | -   | -   | 4     |
| 10.1.4.0    | 10.1.10.2 | 255.255.255.0   | UG    | -      | -   | -   | 4     |
| 10.1.5.0    | 10.1.10.2 | 255.255.255.0   | UG    | -      | -   | -   | 4     |
| 10.1.6.0    | 10.1.10.2 | 255.255.255.0   | UG    | -      | -   | -   | 4     |
| 10.1.20.0   | 10.1.20.2 | 255.255.255.0   | UG    | -      | -   | -   | 5     |
| 10.1.7.0    | 10.1.20.2 | 255.255.255.0   | UG    | -      | -   | -   | 5     |
| 10.1.8.0    | 10.1.20.2 | 255.255.255.0   | UG    | -      | -   | -   | 5     |
| 10.1.7.0    | 10.1.20.2 | 255.255.255.0   | UG    | -      | -   | -   | 5     |
| 10.1.8.0    | 10.1.20.2 | 255.255.255.0   | UG    | -      | -   | -   | 5     |


Node: 9, Time: +0s, Local time: +0s, Ipv4ListRouting table
  Priority: 0 Protocol: ns3::Ipv4StaticRouting
Node: 9, Time: +0s, Local time: +0s, Ipv4StaticRouting table
```

Observations :

- The output shows three active routers Nodes 0, 1, and 2, each maintaining both static as well as Global IPv4 routing tables.
- Node 0 operates as the central routing hub, with Nodes 1 and 2 correctly pointing to Node 0s interfaces as their gateways.

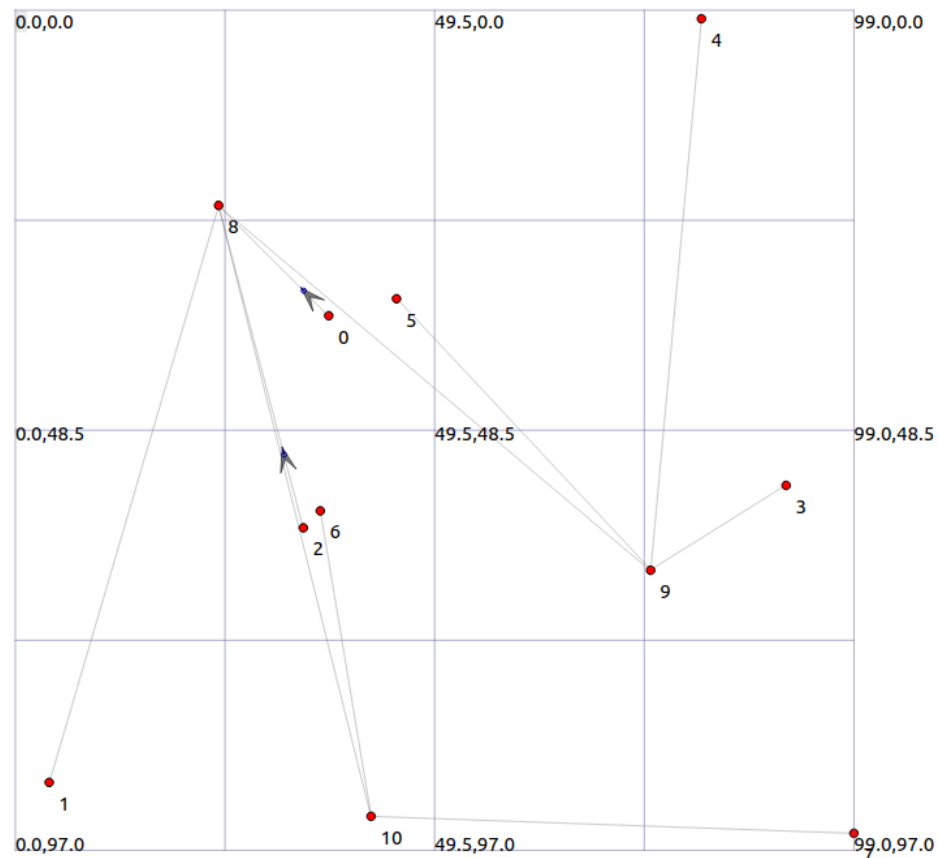
Inferences :

- The network has successfully converged demonstrated by the fully populated Ipv4GlobalRouting tables which guarantee reachability among all defined subnets.
- The consistent presence of the (Up, Gateway) flag on the routing entries confirms that the routers are correctly set up for forwarding multi hop traffic to the intended IP addresses.

Question 5 (as netanim.png):

COMMAND USED :

Inside NetAnim Directory:
cmake . & make & cd bin
./netanim



Observation :

- Topology Rendering: The visualizer maps the complete custom topology, displaying all 11 simulated entities specifically, the 8

terminal nodes and the 3 core routers distributed across the coordinate grid.

- Link and Traffic Visualization: The established PointToPointHelper links are represented as solid lines connecting the edge nodes to the routers, and the routers to each other.

Inferences :

- The spatial layout confirms that our cc node creation and point to point link configurations successfully generated a fully connected, multi hop network.
- The directional blue arrows verify that our configured TCP and UDP OnOffHelper applications are successfully running and actively transmitting packet flows.

Part B :

In this part of the assignment (implemented in /PartB/as6_que2.cc), we advance our custom multi router topology to observe how varying error rates impact overall network reliability. We use dynamic command line parameters to inject channel errs, implement UDP echo applications to generat traffic, and evaluate the performance via the Packet Delivery Ratio (PDR).

1. Simulation Control & Randomization

- The CommandLine module is set up to parse user defined arguments directly into the script, specifically seedr (for RNG), errorRate, and errorUnit.
- To ensure automated test runs do not yield identical pseudorandom sequences, we initialize the random number generator using RngSeedManager::SetSeed(1) and dynamically update the run sequence with RngSeedManager::SetRun(seedr).

2. Node Generation & Link Setup

- The network is populated with 8 end devices (termi.Create(8)) and 3 routing nodes (routers.Create(3)).
- We use dedicated PointToPointHelper objects (alinks, rout0_1, rout0_2) to define connection characteristics. Capacity and latency are fine tuned using SetDeviceAttribute("DataRate", ...) and SetChannelAttribute("Delay", ...), giving access links 10Mbps/2ms profiles and router links narrower bottlenecks (e.g., 5Mbps/5ms and 3Mbps/10ms).

3. Dynamic Error Injection

- We simulate packet corruption by creating a RateErrorModel.
- The code dynamically maps the user's errorUnit input (packet, bit, or byte) to ns3 specific string variables (e.g., ERROR_UNIT_PACKET).

- All relevant NetDevices are grouped into an allErrorDevices container. A loop iterates through this container, attaching the error model to the ReceiveErrorModel attribute of the receiving interfaces.

4. Protocol Stack

- The complete IPv4 network suite is installed on all termi and routers nodes using InternetStackHelper.
- Ipv4AddressHelper handles addressing. Access link subnets are dynamically generated in a loop (formatting strings like 10.1.X.0), while router links are assigned to dedicated static subnets (10.1.10.0 and 10.1.20.0).
- The network learns paths automatically via Ipv4GlobalRoutingHelper::PopulateRoutingTables().

5. UDP Traffic Generation

- Communication flows are established using UdpEchoServerHelper and UdpEchoClientHelper. The specific server client node pairings are cleanly managed using corresponding index arrays (snodes and cnodes).
- Destination IPs are dynamically fetched using acsinterfaces.GetAddress(...). Each of the four clients is explicitly configured to fire 100 packets (MaxPackets), sized at 1024 bytes (PacketSize), at intervals of 0.05 seconds.

6. Performance Tracking & Data Extraction

- FlowMonitorHelper is deployed (monitorflow.InstallAll()) to observe packet behavior across the entire topology.

Metric Calculation: The script loops through monitor->GetFlowStats(), tallying txPackets (transmitted) and rxPackets (received) to compute the final Packet Delivery Ratio.

- **Console Output:** The result is formatted and printed (std::cout << "PDR_RESULT:" << pdr) so that an external automation/Bash script can easily grep the data points.

Packet Delivery Ratio (PDR)

Packet Delivery Ratio is a core network performance metric. It acts as a direct indicator of connection reliability by measuring the exact fraction of data packets that successfully survive the journey from the source to the destination. Tracking this allows us to mathematically quantify how the injected RateErrorModel degrades the channel.

Formula Used: $PDR = \text{Total Packets Received} / \text{Total Packets Transmitted}$

Running the sample commands for code as mentioned in assignment :

1. COMMAND USED :

`./ns3 run "scratch/as6_que2 --PrintAttributes=ns3::RateErrorModel"`

Output Terminal :

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VW:~/ns-allinone-3.44/ns-3.44$ ./ns3 run "scratch/as6_que2 --PrintAttributes=ns3::RateErrorModel"
Consolidate compiler generated dependencies of target scratch_as6_que2
Attributes for TypeId ns3::RateErrorModel
--ns3::RateErrorModel::ErrorRate=[0]
  The error rate.
--ns3::RateErrorModel::ErrorUnit=[ERROR_UNIT_BYTE]
  The error unit.
--ns3::RateErrorModel::RanVar=[ns3::UniformRandomVariable[Min=0.0|Max=1.0]]
  The decision variable attached to this error model.
Attributes defined in parent class ns3::ErrorModel
--ns3::ErrorModel::IsEnabled=[true]
  Whether this ErrorModel is enabled or not.
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VW:~/ns-allinone-3.44/ns-3.44$
```

2.COMMAND USED :

`./ns3 run "scratch/as6_que2 --PrintAttributes=ns3::BurstErrorModel"`

Output Terminal :

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VW:~/ns-allinone-3.44/ns-3.44$ ./ns3 run "scratch/as6_que2 --PrintAttributes=ns3::BurstErrorModel"
Attributes for TypeId ns3::BurstErrorModel
--ns3::BurstErrorModel::BurstSize=[ns3::UniformRandomVariable[Min=1|Max=4]]
  The number of packets being corrupted at one drop.
--ns3::BurstErrorModel::BurstStart=[ns3::UniformRandomVariable[Min=0.0|Max=1.0]]
  The decision variable attached to this error model.
--ns3::BurstErrorModel::ErrorRate=[0]
  The burst error event.
Attributes defined in parent class ns3::ErrorModel
--ns3::ErrorModel::IsEnabled=[true]
  Whether this ErrorModel is enabled or not.
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VW:~/ns-allinone-3.44/ns-3.44$
```


3.COMMAND USED :

`./ns3 run "scratch/as6_que2 --PrintAttributes=ns3::PointToPointNetDevice"`

Output Terminal :

```
whether this ErrorModel is enabled or not.
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/ns-allinone-3.44/ns-3.44$ ./ns3 run "scratch/as6_que2 --PrintAttributes=ns3::PointToPointNetDevice"
Attributes for TypeId ns3::PointToPointNetDevice
--ns3::PointToPointNetDevice:Address=[ff:ff:ff:ff:ff:ff]
  The MAC address of this device.
--ns3::PointToPointNetDevice:DataRate=[32768bps]
  The default data rate for point to point links
--ns3::PointToPointNetDevice:InterframeGap=[+0ns]
  The time to wait between packet (frame) transmissions
--ns3::PointToPointNetDevice:Mtu=[1500]
  The MAC-level Maximum Transmission Unit
--ns3::PointToPointNetDevice:ReceiveErrorModel=[0]
  The receiver error model used to simulate packet loss
--ns3::PointToPointNetDevice:TxQueue=[0]
  A queue to use as the transmit queue in the device.
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/ns-allinone-3.44/ns-3.44$
```

Inferences :

- ns3 handles packet loss ,corruption exclusively at the destination's interface, rather than modifying the transmission channel itself.
- The simulation assumes a completely flawless network environment unless we actively configure and attach an error model.
- Error models in NS3 utilize built in uniform random variables to introduce realistic, unpredictable variance into our network.
- The --PrintAttributes command exposes the exact internal string paths and default values (like a 32768bps data rate) used by ns3

Shell Script for the simulations :

```
Welcome $ runner.sh x
$ runner.sh
1  #!/bin/bash
2  #ROLL NO 12340370 so last two digit as 70 , hardcoded 70 in .cc in case not specified from the terminal
3  BASE_SEED=70
4  MAX_SEED=$((BASE_SEED + 9))
5
6  ERR_RATES=("0.0" "0.2" "0.4" "0.6" "0.8" "1.0")
7  ERR_UNITS=("Packet" "Bit" "Byte")
8
9  if [ ! -x "./ns3" ]; then
10     echo "./ns3 not found. Run from the ns3 directory."
11     exit 1
12 fi
13
14 for u in "${ERR_UNITS[@]}; do
15     outfile="outputdata${u}.dat"
16     echo "ErrorRate AvgPDR" > "$outfile"
17
18     echo "Starting Error Unit: $u"
19
20     for r in "${ERR_RATES[@]}; do
21         sum_pdr=0
22
23         for s in $(seq $BASE_SEED $MAX_SEED); do
24             out=$(./ns3 run "scratch/as6_que2 --errorRate=$r --errorUnit=$u --seedRun=$s" 2>&1)
25
26             pdr_val=$(echo "$out" | grep "PDR_RESULT:" | cut -d':' -f2)
27
28             if [ -z "$pdr_val" ]; then
29                 pdr_val=0
30             fi
31
32             sum_pdr=$(awk "BEGIN {print $sum_pdr + $pdr_val}")
33
34             echo "Unit: $u | Rate: $r | Seed: $s | PDR: $pdr_val"
35         done
36
37         avg_pdr=$(awk "BEGIN {print $sum_pdr / 10}")
38         echo "$r $avg_pdr" >> "$outfile"
39         echo "Average PDR for $r ($u): $avg_pdr"
40         echo "-----"
41     done
42 done
43
44 echo "All simulations complete. Data saved to .dat files."
```

To streamline the data collection process for my as6_que2.cc simulation, I wrote this Bash script (runner.sh). Instead of manually running the code for every parameter change, I designed this script to automatically iterate through all network conditions, compute the average Packet Delivery Ratio (PDR), and generate data files (outputdataPacket.dat, outputdataBit.dat, and outputdataByte.dat) that are fully prepped for Gnuplot visualization.

- I derived a base seed (70) from my roll number to guarantee exactly 10 independent statistical runs for every test case.
- I configured nested loops to systematically test all combinations of error units and rates while generating .dat files for each.

- I passed dynamic parameters into the ns3 command line and used grep and cut to instantly extract the exact PDR value from the terminal output.
- I utilized awk to keep a running PDR total while echoing live execution status to the terminal for real time monitoring.
- I calculated the final 10 run average PDR using awk and appended it as a clean X Y coordinate into the respective .dat file for graphing.

Output Terminal :

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/ns-allinone-3.44/ns-3.44$ ./runner.sh
Starting Error Unit: Packet
Unit: Packet | Rate: 0.0 | Seed: 70 | PDR: 1
Unit: Packet | Rate: 0.0 | Seed: 71 | PDR: 1
Unit: Packet | Rate: 0.0 | Seed: 72 | PDR: 1
Unit: Packet | Rate: 0.0 | Seed: 73 | PDR: 1
Unit: Packet | Rate: 0.0 | Seed: 74 | PDR: 1
Unit: Packet | Rate: 0.0 | Seed: 75 | PDR: 1
Unit: Packet | Rate: 0.0 | Seed: 76 | PDR: 1
Unit: Packet | Rate: 0.0 | Seed: 77 | PDR: 1
Unit: Packet | Rate: 0.0 | Seed: 78 | PDR: 1
Unit: Packet | Rate: 0.0 | Seed: 79 | PDR: 1
Average PDR for 0.0 (Packet): 1
-----
Unit: Packet | Rate: 0.2 | Seed: 70 | PDR: 0.47079
Unit: Packet | Rate: 0.2 | Seed: 71 | PDR: 0.482173
Unit: Packet | Rate: 0.2 | Seed: 72 | PDR: 0.490756
Unit: Packet | Rate: 0.2 | Seed: 73 | PDR: 0.501672
Unit: Packet | Rate: 0.2 | Seed: 74 | PDR: 0.47079
Unit: Packet | Rate: 0.2 | Seed: 75 | PDR: 0.511551
Unit: Packet | Rate: 0.2 | Seed: 76 | PDR: 0.462712
Unit: Packet | Rate: 0.2 | Seed: 77 | PDR: 0.513245
Unit: Packet | Rate: 0.2 | Seed: 78 | PDR: 0.468966
Unit: Packet | Rate: 0.2 | Seed: 79 | PDR: 0.506645
Average PDR for 0.2 (Packet): 0.487929
-----
Unit: Packet | Rate: 0.4 | Seed: 70 | PDR: 0.196653
Unit: Packet | Rate: 0.4 | Seed: 71 | PDR: 0.171739
Unit: Packet | Rate: 0.4 | Seed: 72 | PDR: 0.171975
Unit: Packet | Rate: 0.4 | Seed: 73 | PDR: 0.210417
Unit: Packet | Rate: 0.4 | Seed: 74 | PDR: 0.213552
Unit: Packet | Rate: 0.4 | Seed: 75 | PDR: 0.1875
Unit: Packet | Rate: 0.4 | Seed: 76 | PDR: 0.184874
Unit: Packet | Rate: 0.4 | Seed: 77 | PDR: 0.187764
Unit: Packet | Rate: 0.4 | Seed: 78 | PDR: 0.217659
Unit: Packet | Rate: 0.4 | Seed: 79 | PDR: 0.191579
Average PDR for 0.4 (Packet): 0.193371
-----
Unit: Packet | Rate: 0.6 | Seed: 70 | PDR: 0.0591017
Unit: Packet | Rate: 0.6 | Seed: 71 | PDR: 0.0589623
Unit: Packet | Rate: 0.6 | Seed: 72 | PDR: 0.0431655
Unit: Packet | Rate: 0.6 | Seed: 73 | PDR: 0.0386473
Unit: Packet | Rate: 0.6 | Seed: 74 | PDR: 0.0635294
Unit: Packet | Rate: 0.6 | Seed: 75 | PDR: 0.0633803
Unit: Packet | Rate: 0.6 | Seed: 76 | PDR: 0.0498812
Unit: Packet | Rate: 0.6 | Seed: 77 | PDR: 0.0567376
Unit: Packet | Rate: 0.6 | Seed: 78 | PDR: 0.0522565
Unit: Packet | Rate: 0.6 | Seed: 79 | PDR: 0.05
Average PDR for 0.6 (Packet): 0.053566
-----
Unit: Packet | Rate: 0.8 | Seed: 70 | PDR: 0.00744417
Unit: Packet | Rate: 0.8 | Seed: 71 | PDR: 0.00497512
Unit: Packet | Rate: 0.8 | Seed: 72 | PDR: 0.00497512
Unit: Packet | Rate: 0.8 | Seed: 73 | PDR: 0.00249377
Unit: Packet | Rate: 0.8 | Seed: 74 | PDR: 0.0123457
Unit: Packet | Rate: 0.8 | Seed: 75 | PDR: 0.0123457
```

Gnuplot Code :

```
graphs_que2.gnuplot
1  set terminal pngcairo size 700,600 |
2  set xlabel 'Error Rate'
3  set ylabel 'Average PDR'
4  set yrange [0:1.1]
5  set grid
6
7  set output 'erbit.png'
8  set title 'Average PDR Vs Error Rate (Bit)'
9  set style line 2 lc rgb '#FFA500' lt 1 lw 3 pt 7 ps 1.5
10 plot 'outputdataBit.dat' with linespoints ls 2 title 'Avg PDR'
11
12 set output 'erbyte.png'
13 set title 'Average PDR Vs Error Rate (Byte)'
14 set style line 3 lc rgb '#A52A2A' lt 1 lw 3 pt 9 ps 1.5
15 plot 'outputdataByte.dat' with linespoints ls 3 title 'Avg PDR'
16
17 set output 'erpacket.png'
18 set title 'Average PDR Vs Error Rate (Packet)'
19 set style line 1 lc rgb '#800080' lt 1 lw 3 pt 5 ps 1.5
20 plot 'outputdataPacket.dat' with linespoints ls 1 title 'Avg PDR'
21
22
```

To visualize how different error models impact network reliability then wrote a Gnuplot script (graphs_que2.gnuplot) to process the simulation data into clear line graphs. I configured the global layout with a 700x600 PNG resolution, enabled a background grid, and fixed the Yaxis range from 0 to 1.1 to standardize the average PDR scale. The script systematically reads the outputdataBit.dat, outputdataByte.dat, and outputdataPacket.dat files to generate three corresponding output images (erbit.png, erbyte.png, and erpacket.png). For each plot, I applied distinct line styles including specific hex colors, line weights, and point types to visually differentiate the Error Rate versus Average PDR trends across the bit, byte, and packet error units.

The PDR (Packet Delivery Ratio) tables show the results from the C++ code, which transmitted over 100 packets for each of the three error units (bit, byte, and packet) across different random number generator runs (RngRun).

(i) Packet Error Unit :

RngRun/Error Rate	ER1=0.2	ER2=0.4	ER3=0.6	ER4=0.8	ER5=1.0
RngRun=70	0.4708	0.1967	0.0591	0.0074	0.0000
RngRun=71	0.4822	0.1717	0.0590	0.0050	0.0000
RngRun=72	0.4908	0.1720	0.0432	0.0050	0.0000
RngRun=73	0.5017	0.2104	0.0386	0.0025	0.0000
RngRun=74	0.4708	0.2136	0.0635	0.0123	0.0000
RngRun=75	0.5116	0.1875	0.0634	0.0123	0.0000
RngRun=76	0.4627	0.1849	0.0499	0.0099	0.0000
RngRun=77	0.5132	0.1878	0.0567	0.0050	0.0000
RngRun=78	0.4690	0.2177	0.0523	0.0074	0.0000
RngRun=79	0.5066	0.1916	0.0500	0.0074	0.0000
AVG. PDR	0.4879	0.1934	0.0536	0.0074	0.0000

Observation :

- The Average PDR decreases sharply from 0.4879 at an Error Rate of 0.2 to a complete failure of 0.00 at an Error Rate of 1.0.
- The PDR values for any single Error Rate, such as 0.2, exhibit a range of results across the different RngRun values (e.g., 0.4627 to 0.5132).

Inference:

- The inverse relationship strongly suggests the network's reliability is highly sensitive to the magnitude of the injected packet-level error rate.
- The variation across RngRuns validates the statistical independence , non deterministic nature of the random error model used in the simulation.

(ii) Bit Error Unit :

RngRun/Error Rate	ER1=0.2	ER2=0.4	ER3=0.6	ER4=0.8	ER5=1.0
RngRun=70	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=71	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=72	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=73	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=74	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=75	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=76	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=77	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=78	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=79	0.0000	0.0000	0.0000	0.0000	0.0000
AVG. PDR	0.0000	0.0000	0.0000	0.0000	0.0000

Observation:

- The PDR for all RngRuns is 0.0000 across all tested Error Rates (0.2 to 1.0).

- There is no variation in PDR across the different RngRun values for any given Error Rate, as all recorded values are 0.0000.

Inferences

- The introduction of any non zero Bit Error Rate (0.2 or higher) results in an immediate and complete failure of data transmission (0 PDR).
- This suggests that even a small rate of bit errors is sufficient to corrupt all packets, indicating a high sensitivity of the system to bit level corruption.

(iii) Byte Error Unit :

RngRun/Error Rate	ER1=0.2	ER2=0.4	ER3=0.6	ER4=0.8	ER5=1.0
RngRun=70	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=71	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=72	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=73	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=74	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=75	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=76	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=77	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=78	0.0000	0.0000	0.0000	0.0000	0.0000
RngRun=79	0.0000	0.0000	0.0000	0.0000	0.0000
AVG. PDR	0.0000	0.0000	0.0000	0.0000	0.0000

Observation :

- The PDR for all RngRuns is 0.0000 across all tested Error Rates (0.2 to 1.0).

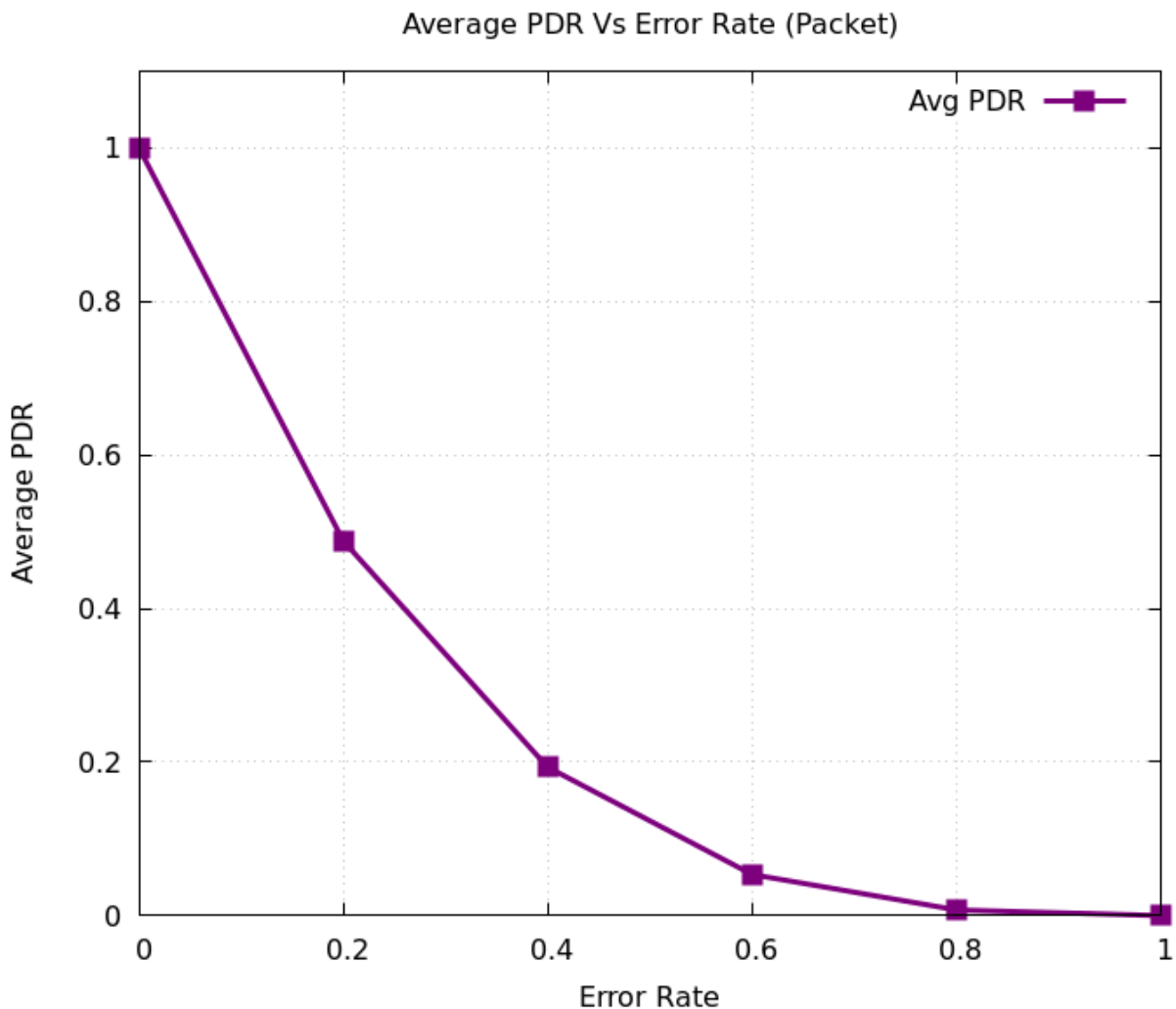
- There is zero variation in PDR across the different RngRun values for any given non zero Error Rate.

Inference :

- The introduction of any non zero Byte Error Rate (0.2 or higher) results in an immediate and complete failure of data transmission (0 PDR).
- This suggests that even a small rate of byte errors is sufficient to corrupt all packets, indicating a high sensitivity of the system to byte level corruption.

Graphs for the Average PDR vs Error Rate for different Units :

(i) Avg. PDR vs Error Rate for Packet error unit (/PartB/erpacket.png):



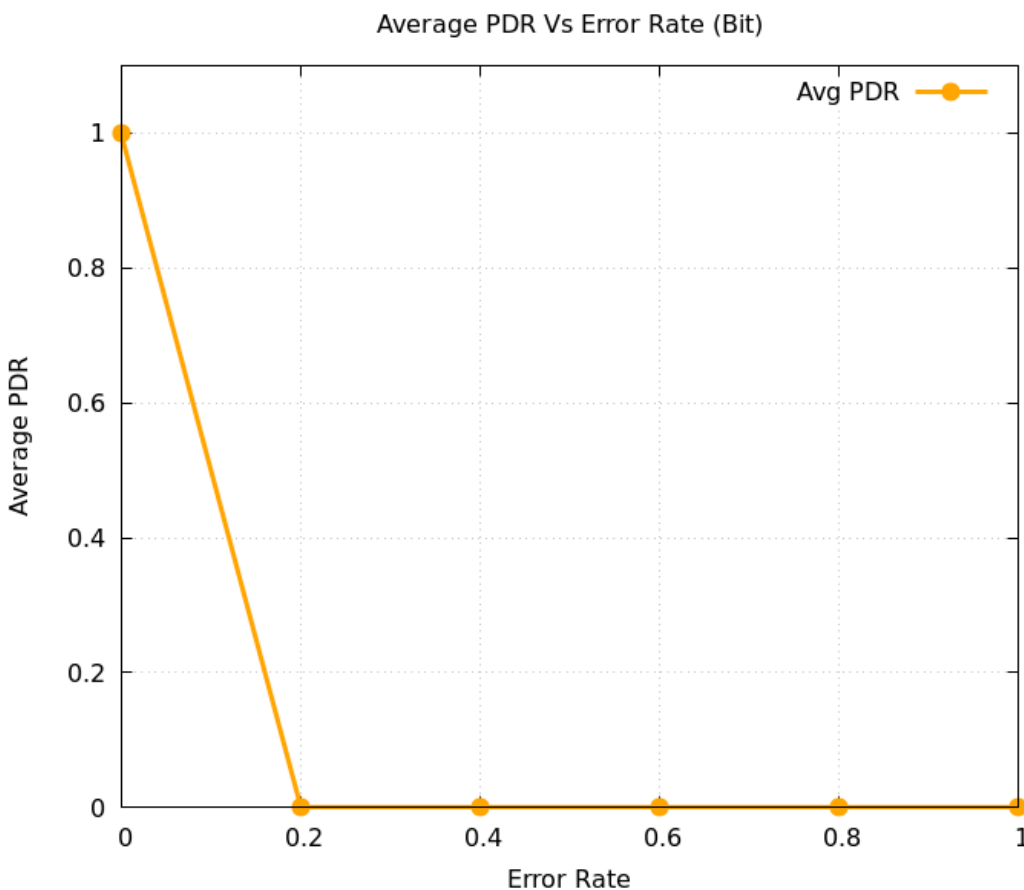
Observations :

- The network loses over half its packets instantly dropping to about 0.48 PDR at just a 0.2 error rate.
- Unlike bit or byte models, the data shows slight PDR fluctuations across different random seeds at identical error rates.

Inferences :

- These fluctuations confirm the "Packet" model is successfully using the random number generator to vary which specific packets drop each run.
- A 50% delivery failure from merely a 20% error rate proves the network is highly sensitive to minor packet loss.

(ii) Avg. PDR vs Error Rate for Bit error unit (/PartB/erbit.png):



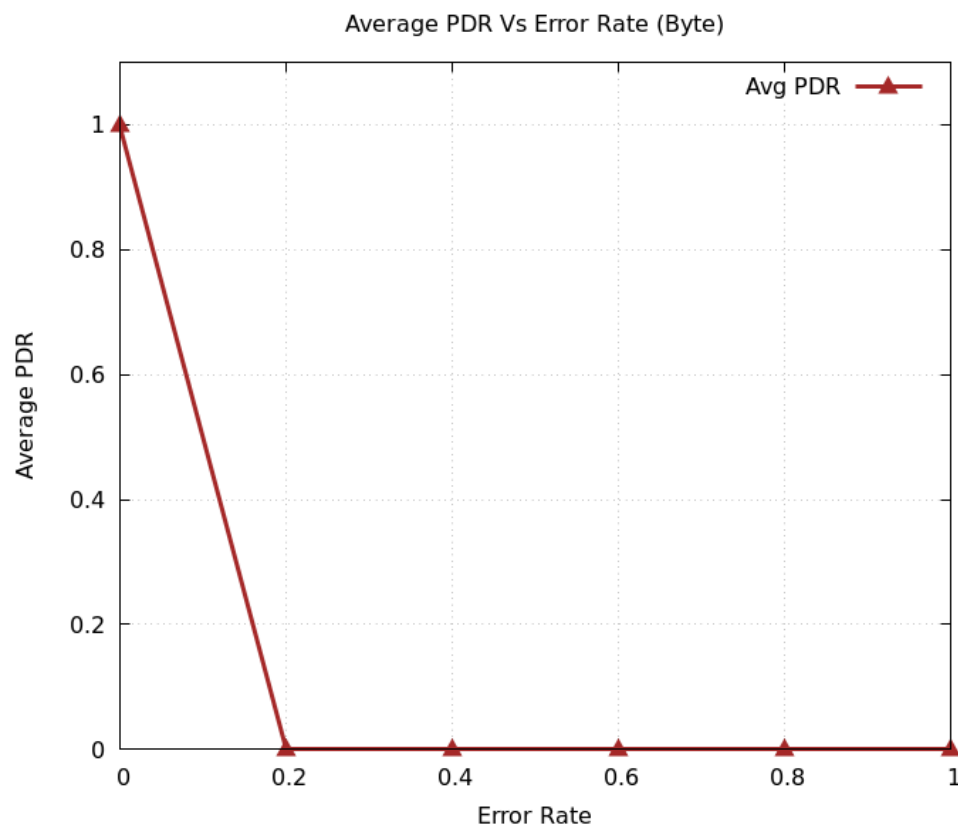
Observation :

- As soon as the error rate increases to the very first increment (0.2), the Average PDR plummets immediately and completely to 0.0.
- For all subsequent, higher error rates (0.4, 0.6, 0.8, and 1.0), the network never recovers; the PDR remains completely flat at 0.

Inferences :

- An error rate of 0.2 means there is a 20% probability that any individual bit will be corrupted during transmission.
- If even a single bit in a packet is altered, the entire packet fails the integrity check and is dropped by the receiving interface

(iii) Avg. PDR vs Error Rate for Bit error unit (/PartB/erbit.png):



Observation :

- Average PDR crashes directly to 0.0 the moment the error rate reaches 0.2.
- The network completely fails to recover at any higher error rate, keeping the PDR flat at 0.

Inferences :

- For a 1024 byte packet, a 20% byte error rate statistically guarantees massive corruption within every single transmitted packet.
- because receiving interfaces drop any packet with even a single corrupted byte due to standard checksum failures, a 0.2 byte error rate effectively destroys 100% of the traffic.